LAPORAN PROYEK 2 EXECUTIVE SUMMARY
DETEKSI PENYAKIT JANTUNG MENGGUNAKAN REGRESI LOGISTIK BINER DAN SUPPORT VECTOR MACHINE

Approved
14/08/2026 09:24

Kelompok 2 :

1. Citra Tri Setyaningrum (V3424005)
2. Zahra Raufatul Azizah (V3424018)
3. Alya Nabila Azhara (V3424019)
4. Gilang Pamungkas (V3424027)
5. Lailla Nurulita Ramadhani (V3424030)
6. Nisrina Khalisha Salsabila (V3424033)

PROGRAM STUDI D3 TEKNIK INFORMATIKA
FAKULTAS SEKOLAH VOKASI
UNIVERSITAS SEBELAS MARET
SURAKARTA

2025

Laporan Progres : 4 Desember 2025

A. Regresi Logistik Biner

1. Sudah menentukan variabel respon (Y) yaitu Heart Disease dan variabel Prediktor (X) yaitu Age, Gender, Blood Pressure, Smoking, Diabetes, Obesity, dan Blood Sugar. Variabel Y berupa Kategori, sehingga analisis regresi yang digunakan yaitu Logistik Biner.
2. Sudah melakukan analisis regresi logistik biner, dengan pembagian data train 70%, 80%, dan 50%
   a. Mengubah data dalam kolom kategorik menjadi berupa angka.

```
> # Mengubah variabel kategori ke bentuk angka
> data$Gender <- ifelse(data$Gender == "Male", 0, 1)
> data$Smoking <- factor(data$Smoking,
+                        levels = c("Never", "Former", "Current"),
+                        ordered = TRUE)
> data$Smoking <- as.numeric(data$Smoking)
> data$Diabetes <- ifelse(data$Diabetes == "No", 0, 1)
> data$Obesity <- ifelse(data$Obesity == "No", 0, 1)
> data
```

	Age	Gender	Cholesterol	Blood.Pressure	Smoking	Exercise.Hours	Diabetes	Obesity	Blood.Sugar	Heart.Disease
1	75	1	228	119	3	1	0	1	119	1
2	48	0	204	165	3	5	0	0	70	0
3	53	0	234	91	1	3	0	1	196	1
4	69	1	192	90	3	4	1	0	107	0
5	62	1	172	163	1	6	1	0	183	0
6	77	0	309	110	1	0	1	1	122	1
7	64	1	211	105	2	8	1	1	120	1
8	60	1	208	148	1	4	1	1	113	1
9	37	1	317	137	3	3	1	1	114	0
10	63	0	204	141	2	8	1	0	107	1
11	67	1	282	108	1	3	0	1	85	1
12	37	0	293	148	3	6	0	1	129	0
13	67	0	325	177	1	8	0	1	192	1
14	43	0	155	169	3	8	1	0	163	0
15	60	0	226	168	1	8	1	0	97	1

   b. Mengubah variabel kategori menjadi faktor

```
> View(data)
> # Mengubah variabel kategori menjadi faktor
> data$Heart.Disease <- as.factor(data$Heart.Disease) #0=No, 1=Yes
> data$Smoking <- as.factor(data$Smoking)
> str(data)
'data.frame':   1000 obs. of  10 variables:
 $ Age           : int  75 48 53 69 62 77 64 60 37 63 ...
 $ Gender        : num  1 0 0 1 1 0 1 1 1 0 ...
 $ Cholesterol   : int  228 204 234 192 172 309 211 208 317 204 ...
 $ Blood.Pressure: int  119 165 91 90 163 110 105 148 137 141 ...
 $ Smoking       : Factor w/ 3 levels "1","2","3": 3 3 1 3 1 1 2 1 3 2 ...
 $ Exercise.Hours: int  1 5 3 4 6 0 8 4 3 8 ...
 $ Diabetes      : num  0 0 0 1 1 1 1 1 1 1 ...
 $ Obesity       : num  1 0 1 0 0 1 1 1 1 0 ...
 $ Blood.Sugar   : int  119 70 196 107 183 122 120 113 114 107 ...
 $ Heart.Disease : Factor w/ 2 levels "0","1": 2 1 2 1 1 2 2 2 1 2 ...
```

   c. Membagi data menjadi data train dan data test

1) Data train 80% dan data test 20%

```
> #Split data training testing
> library(dplyr)
> set.seed(303)
> intrain <- sample(nrow(data), nrow(data)*0.8)
> data_train <- data[intrain,]
> data_test <- data[-intrain,]
> data$Heart.Disease %>%
+   levels()
[1] "0" "1"
```

Data Train

	Age	Gender	Cholesterol	Blood.Pressure	Smoking	Exercise.Hours	Diabetes	Obesity	Blood.Sugar	Heart.Disease
681	75	1	328	111	2	5	1	1	95	1
602	36	0	265	90	2	8	1	1	105	0
622	54	1	287	95	1	9	1	1	94	1
96	46	0	290	149	1	6	1	0	157	0
845	59	1	178	110	2	7	0	1	89	0
536	38	1	340	176	3	5	0	0	155	0
606	36	1	258	131	1	6	0	0	112	0
946	76	0	216	139	3	2	0	1	182	1
615	34	1	313	166	3	5	1	1	86	0
85	62	1	205	127	1	3	1	1	127	1
875	38	1	224	93	2	3	1	1	154	0
641	32	1	269	151	2	9	1	0	199	0
345	49	0	326	157	1	3	1	1	77	0
211	61	0	297	170	3	9	0	0	189	1

Showing 786 to 800 of 800 entries, 10 total columns

Data Test

	Age	Gender	Cholesterol	Blood.Pressure	Smoking	Exercise.Hours	Diabetes	Obesity	Blood.Sugar	Heart.Disease
937	30	0	159	98	3	4	1	0	174	0
938	51	1	218	114	3	2	1	1	176	1
940	65	0	151	107	2	4	1	1	70	0
945	26	1	241	120	3	5	1	1	172	0
957	50	1	227	161	3	9	1	1	97	0
962	41	1	320	159	2	6	1	1	144	0
974	65	1	303	162	1	5	1	1	135	1
983	28	1	243	91	3	8	0	1	138	0
987	69	1	223	147	2	4	1	1	118	1
988	38	1	217	104	1	2	1	1	75	0
989	37	1	349	148	2	5	0	0	144	0
997	78	1	334	145	1	6	0	0	196	1
999	60	1	326	151	2	8	1	0	174	1
1000	53	0	226	116	3	6	0	1	161	1

Showing 186 to 200 of 200 entries, 10 total columns

2) Data train 70% dan data test 30%

```
> #Split data training testing
> library(dplyr)
> set.seed(303)
> intrain <- sample(nrow(data), nrow(data)*0.7)
> data_train <- data[intrain,]
> data_test <- data[-intrain,]
> data$Heart.Disease %>%
+   levels()
[1] "0" "1"
>
```

Data Train

	Age	Gender	Cholesterol	Blood.Pressure	Smoking	Exercise.Hours	Diabetes	Obesity	Blood.Sugar	Heart.Disease
886	28	0	283	139	1	3	0	1	159	0
737	66	1	278	176	3	6	1	1	89	1
433	79	0	222	150	3	6	1	1	71	1
290	67	1	244	124	2	3	1	1	166	1
995	52	0	248	159	2	9	1	1	152	1
975	70	1	253	138	2	0	0	1	179	1
178	68	1	292	167	3	1	0	0	134	1
279	54	1	201	112	1	7	1	0	109	1
265	42	1	278	158	1	4	1	1	140	0
984	27	1	239	143	3	0	1	0	78	0
146	74	1	170	126	2	7	0	0	105	0
414	57	1	229	179	1	2	1	0	144	1
843	61	1	228	124	1	8	0	0	179	1
131	25	0	224	119	1	3	1	1	173	0

Showing 686 to 700 of 700 entries, 10 total columns

Data Test

	Age	Gender	Cholesterol	Blood.Pressure	Smoking	Exercise.Hours	Diabetes	Obesity	Blood.Sugar	Heart.Disease
946	76	0	216	139	3	2	0	1	182	1
947	36	1	341	104	1	0	0	0	179	0
957	50	1	227	161	3	9	1	1	97	0
962	41	1	320	159	2	6	1	1	144	0
972	55	1	219	166	2	5	0	0	113	1
974	65	1	303	162	1	5	1	1	135	1
983	28	1	243	91	3	8	0	1	138	0
986	29	0	224	118	1	1	1	0	145	0
987	69	1	223	147	2	4	1	1	118	1
988	38	1	217	104	1	2	1	1	75	0
989	37	1	349	148	2	5	0	0	144	0
997	78	1	334	145	1	6	0	0	196	1
999	60	1	326	151	2	8	1	0	174	1
1000	53	0	226	116	3	6	0	1	161	1

Showing 286 to 300 of 300 entries, 10 total columns

3) Data train 50% dan data test 50%

```
> #Split data training testing
> library(dplyr)
> set.seed(303)
> intrain <- sample(nrow(data), nrow(data)*0.5)
> data_train <- data[intrain,]
> data_test <- data[-intrain,]
> data$Heart.Disease %>%
+   levels()
[1] "0" "1"
```

Data Train

	Age	Gender	Cholesterol	Blood.Pressure	Smoking	Exercise.Hours	Diabetes	Obesity	Blood.Sugar	Heart.Disease
282	35	0	171	172	2	7	1	0	107	0
800	74	1	184	147	3	7	0	0	149	0
31	32	1	313	154	2	5	1	1	176	0
548	29	1	207	113	1	3	1	1	188	0
948	33	0	212	120	2	9	1	0	150	0
981	79	0	245	132	3	6	0	0	185	1
619	72	1	279	162	3	7	1	1	157	1
338	27	0	211	140	2	8	0	1	95	0
859	51	0	225	170	2	4	1	1	173	1
213	70	0	167	125	3	7	0	0	145	0
828	67	1	296	90	1	9	0	0	123	1
571	77	0	163	91	2	3	1	1	145	0
808	51	0	338	175	1	9	1	1	102	1
766	36	0	189	177	1	5	0	0	113	0

Showing 486 to 500 of 500 entries, 10 total columns

Data Test

	Age	Gender	Cholesterol	Blood.Pressure	Smoking	Exercise.Hours	Diabetes	Obesity	Blood.Sugar	Heart.Disease
972	55	1	219	166	2	5	0	0	113	1
974	65	1	303	162	1	5	1	1	135	1
975	70	1	253	138	2	0	0	1	179	1
980	36	1	161	178	1	7	1	0	185	0
983	28	1	243	91	3	8	0	1	138	0
984	27	1	239	143	3	0	1	0	78	0
986	29	0	224	118	1	1	1	0	145	0
987	69	1	223	147	2	4	1	1	118	1
988	38	1	217	104	1	2	1	1	75	0
989	37	1	349	148	2	5	0	0	144	0
995	52	0	248	159	2	9	1	1	152	1
997	78	1	334	145	1	6	0	0	196	1
999	60	1	326	151	2	8	1	0	174	1
1000	53	0	226	116	3	6	0	1	161	1

Showing 486 to 500 of 500 entries, 10 total columns

d. Membuat model dengan menggunakan semua variabel

1) Data Train 80%

```
> #pembentukan model
> model <- glm( Heart.Disease ~ Age+Gender+Cholesterol+Blood.Pressure+Smoking+Exercise.Hours+Diabetes+Obesity+Blood.Sugar, family = binomial, data
= data_train)
> model

Call:  glm(formula = Heart.Disease ~ Age + Gender + Cholesterol + Blood.Pressure +
    Smoking + Exercise.Hours + Diabetes + Obesity + Blood.Sugar,
    family = binomial, data = data_train)

Coefficients:
   (Intercept)            Age         Gender     Cholesterol  Blood.Pressure        Smoking2        Smoking3  Exercise.Hours
    -21.928714        0.217171       -0.303806        0.035568        0.001948       -0.330983       -0.082991        0.023385
      Diabetes         Obesity     Blood.Sugar
      0.435500       -0.098994        0.001304

Degrees of Freedom: 799 Total (i.e. Null);  789 Residual
Null Deviance:      1079
Residual Deviance: 422.7        AIC: 444.7
>
```

2) Data Train 70%

```
> #pembentukan model
> model <- glm( Heart.Disease ~ Age+Gender+Cholesterol+Blood.Pressure+Smoking+Exercise.Hours+Diabetes+Obesity+Blood.Sugar, family = binomial, data
= data_train)
> model

Call:  glm(formula = Heart.Disease ~ Age + Gender + Cholesterol + Blood.Pressure +
    Smoking + Exercise.Hours + Diabetes + Obesity + Blood.Sugar,
    family = binomial, data = data_train)

Coefficients:
   (Intercept)            Age         Gender     Cholesterol  Blood.Pressure        Smoking2        Smoking3  Exercise.Hours
    -23.069645        0.226794       -0.356197        0.036791        0.002945       -0.328945       -0.151540        0.019078
      Diabetes         Obesity     Blood.Sugar
      0.413001       -0.115947        0.003102

Degrees of Freedom: 699 Total (i.e. Null);  689 Residual
Null Deviance:      945.4
Residual Deviance: 355.6        AIC: 377.6
```

3) Data Train 50%

```
> #pembentukan model
> model <- glm( Heart.Disease ~ Age+Gender+Cholesterol+Blood.Pressure+Smoking+Exercise.Hours+Diabetes+Obesity+Blood.Sugar, family = binomial, data
= data_train)
> model

Call:  glm(formula = Heart.Disease ~ Age + Gender + Cholesterol + Blood.Pressure +
    Smoking + Exercise.Hours + Diabetes + Obesity + Blood.Sugar,
    family = binomial, data = data_train)

Coefficients:
   (Intercept)            Age         Gender     Cholesterol  Blood.Pressure        Smoking2        Smoking3  Exercise.Hours
    -2.222e+01       2.314e-01      -5.580e-01       3.833e-02      -6.982e-04      -3.717e-01      -3.587e-01      -3.830e-03
      Diabetes        Obesity     Blood.Sugar
     2.968e-01      -4.179e-01      -1.085e-03

Degrees of Freedom: 499 Total (i.e. Null);  489 Residual
Null Deviance:      679.6
Residual Deviance: 255.1        AIC: 277.1
>
```

e. Melakukan uji serentak dan uji parsial pada model yang sudah dibuat

1) Data Train 80%. Terlihat nilai G2(656.52) > qchisq(16.92), maka setidaknya ada 1 variabel prediktor (X) berpengaruh. Pada hasil uji parsial diketahui bahwa variabel prediktor yang berpengaruh adalah variabel Age dan Cholesterol.

```
> #Uji serentak
> library(pscl)
> pR2(model)
fitting null model for pseudo-r2
         llh      llhNull           G2     McFadden          r2ML          r2CU
-211.3419069 -539.6023156  656.5208174    0.6083377    0.5598550    0.7560480
> qchisq(0.95,9) #0.95 brt nilai dapat dipercaya(umum), 3 = jumlah variabel-1
[1] 16.91898
> #Uji Parsial
> summary(model)

Call:
glm(formula = Heart.Disease ~ Age + Gender + Cholesterol + Blood.Pressure +
    Smoking + Exercise.Hours + Diabetes + Obesity + Blood.Sugar,
    family = binomial, data = data_train)

Coefficients:
                Estimate Std. Error z value Pr(>|z|)
(Intercept)    -21.928714   1.915358 -11.449   <2e-16 ***
Age              0.217171   0.016408  13.236   <2e-16 ***
Gender          -0.303806   0.250377  -1.213   0.2250
Cholesterol      0.035568   0.003239  10.981   <2e-16 ***
Blood.Pressure   0.001948   0.004631   0.421   0.6741
Smoking2        -0.330983   0.302971  -1.092   0.2746
Smoking3        -0.082991   0.301642  -0.275   0.7832
Exercise.Hours   0.023385   0.043357   0.539   0.5896
Diabetes         0.435500   0.251374   1.732   0.0832 .
Obesity         -0.098994   0.249701  -0.396   0.6918
Blood.Sugar      0.001304   0.003375   0.386   0.6992
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1079.20  on 799  degrees of freedom
Residual deviance:  422.68  on 789  degrees of freedom
AIC: 444.68

Number of Fisher Scoring iterations: 7
```

2) Data Train 70%. Terlihat nilai G2(589.72) > qchisq(16.92), maka setidaknya ada 1 variabel prediktor (X) berpengaruh. Pada hasil uji parsial diketahui bahwa variabel prediktor yang berpengaruh adalah variabel Age dan Cholesterol.

```
> #Uji serentak
> library(pscl)
> pR2(model)
fitting null model for pseudo-r2
         llh      llhNull           G2     McFadden         r2ML         r2CU
-177.8206440 -472.6824826  589.7236772    0.6238053    0.5693512    0.7684642
> qchisq(0.95,9) #0.95 brt nilai dapat dipercaya(umum), 3 = jumlah variabel-1
[1] 16.91898
> #Uji Parsial
> summary(model)

Call:
glm(formula = Heart.Disease ~ Age + Gender + Cholesterol + Blood.Pressure +
    Smoking + Exercise.Hours + Diabetes + Obesity + Blood.Sugar,
    family = binomial, data = data_train)

Coefficients:
                Estimate Std. Error z value Pr(>|z|)
(Intercept)    -23.069645   2.164605 -10.658   <2e-16 ***
Age              0.226794   0.018547  12.228   <2e-16 ***
Gender          -0.356197   0.274551  -1.297    0.195
Cholesterol      0.036791   0.003637  10.116   <2e-16 ***
Blood.Pressure   0.002945   0.005058   0.582    0.560
Smoking2        -0.328945   0.330841  -0.994    0.320
Smoking3        -0.151540   0.330419  -0.459    0.647
Exercise.Hours   0.019078   0.046725   0.408    0.683
Diabetes         0.413001   0.274733   1.503    0.133
Obesity         -0.115947   0.272875  -0.425    0.671
Blood.Sugar      0.003102   0.003709   0.836    0.403
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 945.36  on 699  degrees of freedom
Residual deviance: 355.64  on 689  degrees of freedom
AIC: 377.64

Number of Fisher Scoring iterations: 7
```

3) Data Train 50%. Terlihat nilai G(424.55) >qchisq(16.92), maka setidaknya ada 1 variabel prediktor (X) berpengaruh. Pada hasil uji parsial diketahui bahwa variabel prediktor yang berpengaruh adalah variabel Age dan Cholesterol.

```
> #Uji serentak
> library(pscl)
> pR2(model)
fitting null model for pseudo-r2
         llh      llhNull           G2     McFadden         r2ML         r2CU
-127.5433512 -339.8191198  424.5515372    0.6246728    0.5722015    0.7699642
> qchisq(0.95,9) #0.95 brt nilai dapat dipercaya(umum), 3 = jumlah variabel-1
[1] 16.91898
> #Uji Parsial
> summary(model)

Call:
glm(formula = Heart.Disease ~ Age + Gender + Cholesterol + Blood.Pressure +
    Smoking + Exercise.Hours + Diabetes + Obesity + Blood.Sugar,
    family = binomial, data = data_train)

Coefficients:
                 Estimate Std. Error z value Pr(>|z|)
(Intercept)    -2.222e+01  2.523e+00  -8.805   <2e-16 ***
Age             2.314e-01  2.256e-02  10.255   <2e-16 ***
Gender         -5.580e-01  3.294e-01  -1.694   0.0903 .
Cholesterol     3.833e-02  4.389e-03   8.732   <2e-16 ***
Blood.Pressure -6.982e-04  6.155e-03  -0.113   0.9097
Smoking2       -3.717e-01  3.905e-01  -0.952   0.3411
Smoking3       -3.587e-01  3.980e-01  -0.901   0.3674
Exercise.Hours -3.831e-03  5.436e-02  -0.070   0.9438
Diabetes        2.968e-01  3.261e-01   0.910   0.3628
Obesity        -4.179e-01  3.305e-01  -1.264   0.2061
Blood.Sugar    -1.085e-03  4.541e-03  -0.239   0.8112
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 679.64  on 499  degrees of freedom
Residual deviance: 255.09  on 489  degrees of freedom
AIC: 277.09

Number of Fisher Scoring iterations: 7
```

f. Membuat model dengan hanya menggunakan variabel yang berpengaruh (Age dan Cholesterol)

   1) Data Train 80%. Model yang terbentuk:

```
> #pembentukan model ke-2
> model2 <- glm( Heart.Disease ~ Age+Cholesterol, family = binomial, data = data_train)
> model2

Call:  glm(formula = Heart.Disease ~ Age + Cholesterol, family = binomial,
    data = data_train)

Coefficients:
(Intercept)          Age  Cholesterol
  -21.26740      0.21344      0.03541

Degrees of Freedom: 799 Total (i.e. Null);  797 Residual
Null Deviance:      1079
Residual Deviance: 429  AIC: 435
> summary(model2)

Call:
glm(formula = Heart.Disease ~ Age + Cholesterol, family = binomial,
    data = data_train)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -21.267401   1.608605  -13.22   <2e-16 ***
Age           0.213444   0.015893   13.43   <2e-16 ***
Cholesterol   0.035408   0.003186   11.11   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1079.2  on 799  degrees of freedom
Residual deviance:  429.0  on 797  degrees of freedom
AIC: 435

Number of Fisher Scoring iterations: 7
```

$$\log it\left(\frac{\pi}{1-\pi}\right) = -21.26740 + 0.21344Age + 0.03541Colesterol$$

2) Data Train 70%. Model yang terbentuk:

```
> #pembentukan model ke-2
> model2 <- glm( Heart.Disease ~ Age+Cholesterol, family = binomial, data = data_train)
> model2

Call:  glm(formula = Heart.Disease ~ Age + Cholesterol, family = binomial,
    data = data_train)

Coefficients:
(Intercept)          Age  Cholesterol
  -21.93443      0.22142      0.03624

Degrees of Freedom: 699 Total (i.e. Null);  697 Residual
Null Deviance:      945.4
Residual Deviance: 361.7        AIC: 367.7
> summary(model2)

Call:
glm(formula = Heart.Disease ~ Age + Cholesterol, family = binomial,
    data = data_train)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -21.934434   1.803199  -12.16   <2e-16 ***
Age           0.221420   0.017806   12.44   <2e-16 ***
Cholesterol   0.036236   0.003546   10.22   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 945.36  on 699  degrees of freedom
Residual deviance: 361.67  on 697  degrees of freedom
AIC: 367.67

Number of Fisher Scoring iterations: 7
```

$$\log it\left(\frac{\pi}{1-\pi}\right) = -21.934434 + 0.221420Age + 0.036236Colesterol$$

3) Data train 50%. Model yang terbentuk:

```
> #pembentukan model ke-2
> model2 <- glm( Heart.Disease ~ Age+Cholesterol, family = binomial, data = data_train)
> model2

Call:  glm(formula = Heart.Disease ~ Age + Cholesterol, family = binomial,
    data = data_train)

Coefficients:
(Intercept)          Age  Cholesterol
  -22.49074      0.22439      0.03762

Degrees of Freedom: 499 Total (i.e. Null);  497 Residual
Null Deviance:      679.6
Residual Deviance: 260.9        AIC: 266.9
> summary(model2)

Call:
glm(formula = Heart.Disease ~ Age + Cholesterol, family = binomial,
    data = data_train)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -22.490736   2.181500 -10.310   <2e-16 ***
Age           0.224388   0.021451  10.460   <2e-16 ***
Cholesterol   0.037619   0.004262   8.827   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 679.64  on 499  degrees of freedom
Residual deviance: 260.94  on 497  degrees of freedom
AIC: 266.94

Number of Fisher Scoring iterations: 7
```

$$\log it\left(\frac{\pi}{1-\pi}\right) = -22.490736 + 0.224388Age + 0.037619Colesterol$$

g. Melakukan uji Goodness of Fit dengan Hosmer and Lemeshow

1) Data Train 80%. Nilai p-value(0.1162) > α (0.05), menunjukan bahwa model fit.

```
> #Uji GoF
> library(generalhoslem)
> mod2 <- model.frame(model2)
> suppressWarnings({
+   gof_result <- logitgof(mod2$Heart.Disease, fitted(model2),g=7)
+ })
> print(gof_result)

        Hosmer and Lemeshow test (binary model)

data:  mod2$Heart.Disease, fitted(model2)
X-squared = 8.8271, df = 5, p-value = 0.1162
```

2) Data Train 70%. Nilai p-value(0.1971) > α (0.05), menunjukan bahwa model fit.

```
> #Uji GoF
> library(generalhoslem)
> mod2 <- model.frame(model2)
> suppressWarnings({
+   gof_result <- logitgof(mod2$Heart.Disease, fitted(model2),g=7)
+ })
> print(gof_result)

        Hosmer and Lemeshow test (binary model)

data:  mod2$Heart.Disease, fitted(model2)
X-squared = 7.3316, df = 5, p-value = 0.1971
```

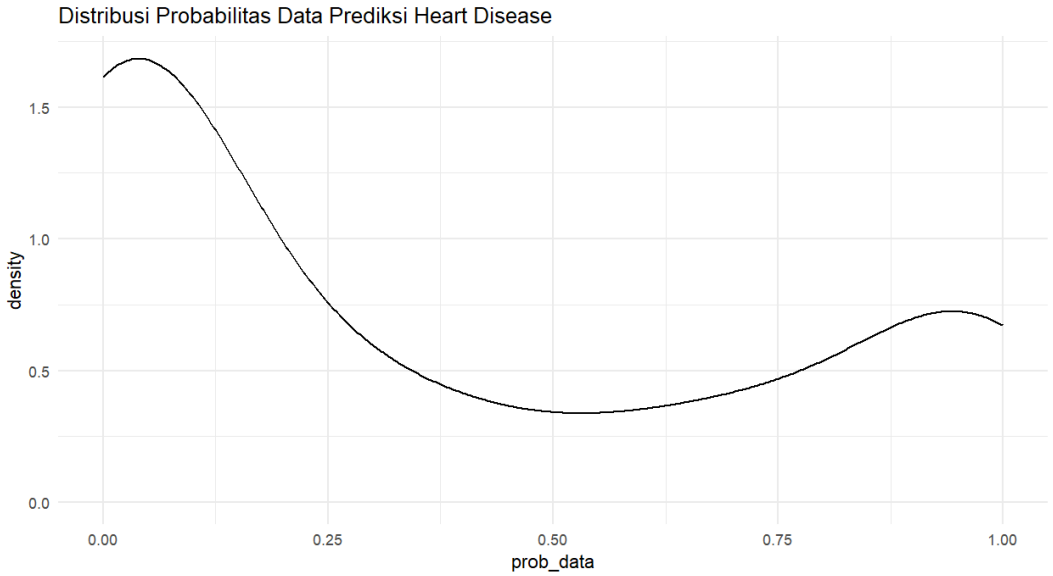
3) Data Train 50%. Nilai p-value(0.1695) > α (0.05), menunjukan bahwa model fit.

```
> #Uji GoF
> library(generalhoslem)
> mod2 <- model.frame(model2)
> suppressWarnings({
+   gof_result <- logitgof(mod2$Heart.Disease, fitted(model2),g=7)
+ })
> print(gof_result)

        Hosmer and Lemeshow test (binary model)

data:  mod2$Heart.Disease, fitted(model2)
X-squared = 7.7673, df = 5, p-value = 0.1695
```

h. Membuat plot distribusi probabilitas data prediksi Heart Disease

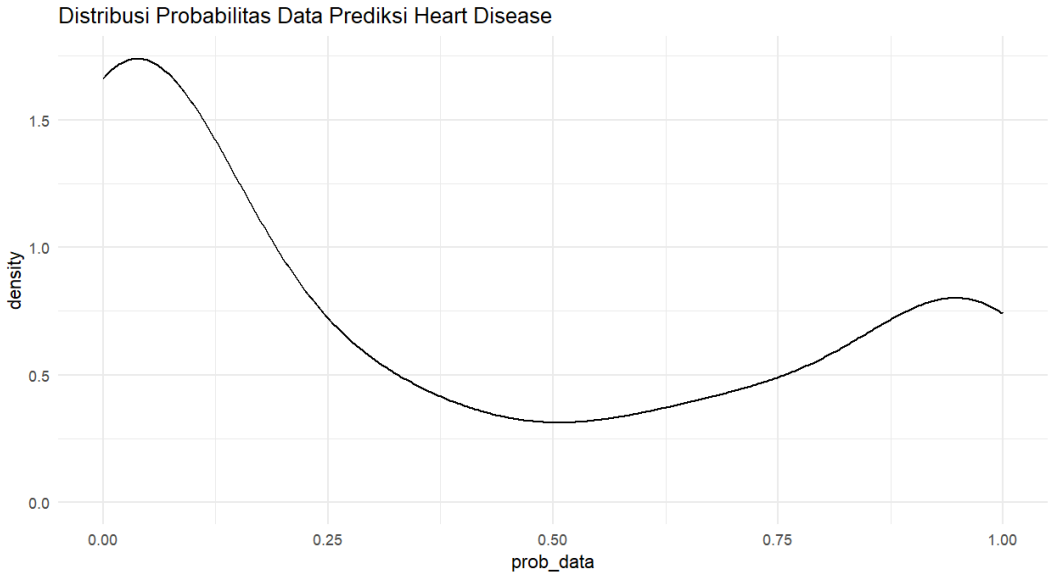
1) Data Train 80%.

```
> #Prediksi
> library(ggplot2)
> data_test$prob_data <- predict(model2, type = "response", newdata = data_test)
> ggplot(data_test, aes(x=prob_data)) +
+   geom_density(lwd=0.5) +
+   labs(title = "Distribusi Probabilitas Data Prediksi Heart Disease") +
+   theme_minimal()
>
```



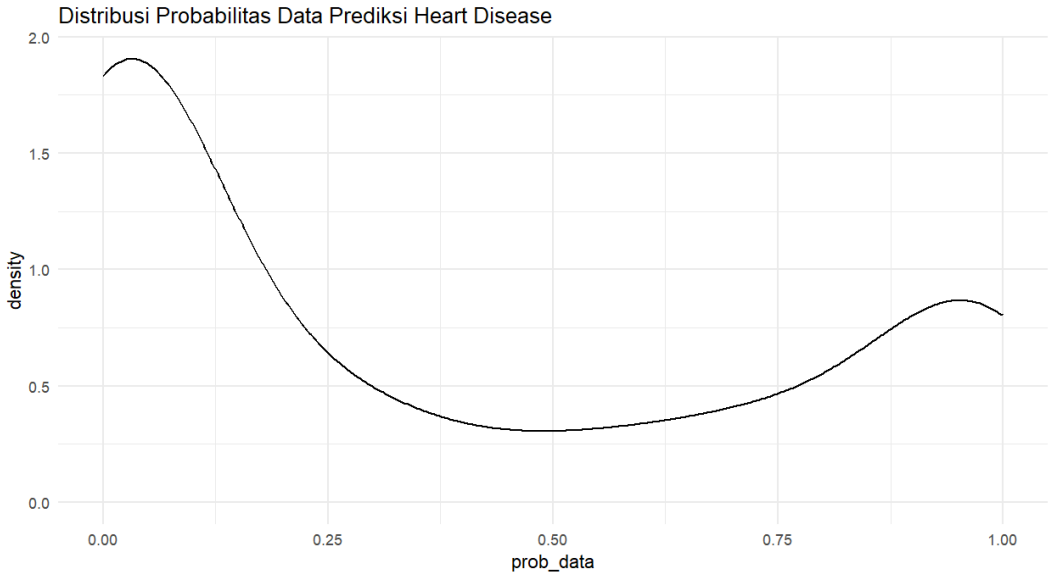
2) Data Train 70%.

```
> #Prediksi
> library(ggplot2)
> data_test$prob_data <- predict(model2, type = "response", newdata = data_test)
> ggplot(data_test, aes(x=prob_data)) +
+   geom_density(lwd=0.5) +
+   labs(title = "Distribusi Probabilitas Data Prediksi Heart Disease") +
+   theme_minimal()
>
```



3) Data Train 50%.

```
> #Prediksi
> library(ggplot2)
> data_test$prob_data <- predict(model2, type = "response", newdata = data_test)
> ggplot(data_test, aes(x=prob_data)) +
+   geom_density(lwd=0.5) +
+   labs(title = "Distribusi Probabilitas Data Prediksi Heart Disease") +
+   theme_minimal()
>
```



i. Mencari nilai threshold optimal

1) Data Train 80%

```
> # Mencari threshold optimal
> library(pROC)
> roc_curve <- roc(data_test$Heart.Disease, data_test$prob_data)
Setting levels: control = 0, case = 1
Setting direction: controls < cases
> optimal_threshold <- coords(roc_curve, "best", ret="threshold")
> print(optimal_threshold)
  threshold
1 0.2853641
> data_test$pred_data <- factor(ifelse(data_test$prob_data > 0.285, "1", "0"))
> data_test[1:10, c("pred_data","Heart.Disease")]
   pred_data Heart.Disease
4          1             0
6          1             1
11         1             1
12         0             0
16         0             0
17         0             0
19         0             0
28         1             1
38         1             1
45         1             1
```

2) Data Train 70%

```
> # Mencari threshold optimal
> library(pROC)
> roc_curve <- roc(data_test$Heart.Disease, data_test$prob_data)
Setting levels: control = 0, case = 1
Setting direction: controls < cases
> optimal_threshold <- coords(roc_curve, "best", ret="threshold")
> print(optimal_threshold)
  threshold
1 0.2828298
> data_test$pred_data <- factor(ifelse(data_test$prob_data > 0.283, "1", "0"))
> data_test[1:10, c("pred_data","Heart.Disease")]
   pred_data Heart.Disease
2          0             0
3          0             1
4          1             0
6          1             1
10         1             1
11         1             1
12         0             0
16         0             0
17         0             0
19         0             0
```

3) Data Train 50%

```
> # Mencari threshold optimal
> library(pROC)
> roc_curve <- roc(data_test$Heart.Disease, data_test$prob_data)
Setting levels: control = 0, case = 1
Setting direction: controls < cases
> optimal_threshold <- coords(roc_curve, "best", ret="threshold")
> print(optimal_threshold)
  threshold
1 0.2596821
> data_test$pred_data <- factor(ifelse(data_test$prob_data > 0.260, "1", "0"))
> data_test[1:10, c("pred_data","Heart.Disease")]
   pred_data Heart.Disease
2          0             0
3          0             1
4          1             0
6          1             1
9          0             0
10         1             1
11         1             1
12         0             0
14         0             0
16         0             0
>
```

j. Melakukan evaluasi model, mencari nilai precision, dan F1-Score

1) Data Train 80%

```
> #Evaluasi Model
> library(caret)
> confusionMatrix(data_test$Heart.Disease, data_test$pred_data)
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 111  20
         1   6  63

               Accuracy : 0.87
                 95% CI : (0.8153, 0.9133)
    No Information Rate : 0.585
    P-Value [Acc > NIR] : < 2e-16

                  Kappa : 0.7255

 Mcnemar's Test P-Value : 0.01079

            Sensitivity : 0.9487
            Specificity : 0.7590
         Pos Pred Value : 0.8473
         Neg Pred Value : 0.9130
             Prevalence : 0.5850
         Detection Rate : 0.5550
   Detection Prevalence : 0.6550
      Balanced Accuracy : 0.8539

       'Positive' Class : 0

> #precision = TP/(TP+FP)
> precision <- 111/(111+20)
> precision
[1] 0.8473282
> #F1 Score = 2*(Sensi*preci)/(sensi+preci)
> F1Score <- 2*(0.9487 *0.8473282)/(0.9487 +0.8473282)
> F1Score
[1] 0.8951533
```

2) Data Train 70%

```
> #Evaluasi Model
> library(caret)
> confusionMatrix(data_test$Heart.Disease, data_test$pred_data)
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 159  33
         1  12  96

               Accuracy : 0.85
                 95% CI : (0.8045, 0.8884)
    No Information Rate : 0.57
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.6878

 Mcnemar's Test P-Value : 0.002869

            Sensitivity : 0.9298
            Specificity : 0.7442
         Pos Pred Value : 0.8281
         Neg Pred Value : 0.8889
             Prevalence : 0.5700
         Detection Rate : 0.5300
   Detection Prevalence : 0.6400
      Balanced Accuracy : 0.8370

       'Positive' Class : 0

> #precision = TP/(TP+FP)
> precision <- 159/(159+33)
> precision
[1] 0.828125
> #F1 Score = 2*(Sensi*preci)/(sensi+preci)
> F1Score <- 2*(0.9298 *0.828125)/(0.9298 +0.828125)
> F1Score
[1] 0.8760222
```

3) Data Train 50%

```
> #Evaluasi Model
> library(caret)
> confusionMatrix(data_test$Heart.Disease, data_test$pred_data)
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 263  54
         1  20 163

               Accuracy : 0.852
                 95% CI : (0.8178, 0.882)
    No Information Rate : 0.566
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.6931

 Mcnemar's Test P-Value : 0.000125

            Sensitivity : 0.9293
            Specificity : 0.7512
         Pos Pred Value : 0.8297
         Neg Pred Value : 0.8907
             Prevalence : 0.5660
         Detection Rate : 0.5260
   Detection Prevalence : 0.6340
      Balanced Accuracy : 0.8402

       'Positive' Class : 0

> #precision = TP/(TP+FP)
> precision <- 263/(263+54)
> precision
[1] 0.829653
> #F1 Score = 2*(Sensi*preci)/(sensi+preci)
> F1Score <- 2*(0.9487 *0.829653)/(0.9487 +0.829653)
> F1Score
[1] 0.8851919
```

k. Mencari nilai odds ration

1) Data Train 80%

```
> #Odds Ratio
> exp(coef(model2))
 (Intercept)          Age  Cholesterol
5.803436e-10 1.237934e+00 1.036042e+00
>
```

Age : Kecenderungan seseorang dengan usia lebih tinggi 1 tahun untuk menderita Heart Disease adalah 1.237934

kali lebih tinggi dibanding dengan yang berusia 1 tahun lebih rendah

Cholesterol : Kecenderungan seseorang dengan level kolesterol 1 mg/dl lebih tinggi untuk menderit aHeart Disease adalah 1.036042 kali lebih tinggi dibanding yang memiliki level kolesterol 1 mg/dl lebih rendah

2) Data Train 70%

```
> #Odds Ratio
> exp(coef(model2))
 (Intercept)          Age  Cholesterol
2.978492e-10 1.247847e+00 1.036901e+00
>
```

Age : Kecenderungan seseorang dengan usia lebih tinggi 1 tahun untuk menderita Heart Disease adalah 1.247847

kali lebih tinggi dibanding dengan yang berusia 1 tahun lebih rendah

Cholesterol : Kecenderungan seseorang dengan level kolesterol 1 mg/dl lebih tinggi untuk menderit aHeart Disease adalah 1.036901 kali lebih tinggi dibanding yang memiliki level kolesterol 1 mg/dl lebih rendah

3) Data Train 50%

```
> #Odds Ratio
> exp(coef(model2))
 (Intercept)          Age  Cholesterol
1.707644e-10 1.251557e+00 1.038335e+00
>
```

Age : Kecenderungan seseorang dengan usia lebih tinggi 1 tahun untuk menderita Heart Disease adalah 1.251557

kali lebih tinggi dibanding dengan yang berusia 1 tahun lebih rendah

Cholesterol : Kecenderungan seseorang dengan level kolesterol 1 mg/dl lebih tinggi untuk menderita Heart Disease adalah 1.038335 kali lebih tinggi dibanding yang memiliki level kolesterol 1 mg/dl lebih rendah

B. Support Vector Machine

1. Mengimport beberapa library yang dibutuhkan
2. Mengimport dataset yang digunakan dan melakukan konversi data yang berupa kategori ke numerik (0/1).

```
import pandas as pd

# Specify the path to your CSV file
csv_file_path = '/content/drive/MyDrive/heart_disease_clean.csv'

# Read the CSV file using pandas, specifying the semicolon separator
df = pd.read_csv(csv_file_path, sep=';')

# --- KONVERSI VARIABEL KATEGORIKAL KE NUMERIK ---
# Binary encoding
df['Gender'] = df['Gender'].map({'Female': 0, 'Male': 1})
df['Diabetes'] = df['Diabetes'].map({'No': 0, 'Yes': 1})
df['Obesity'] = df['Obesity'].map({'No': 0, 'Yes': 1})

# One-hot encoding untuk Smoking (karena punya 3 kategori)
df = pd.get_dummies(df, columns=['Smoking'], prefix='Smoking')

# Convert kolom Smoking dummy (True/False) menjadi 0/1
smoking_cols = [col for col in df.columns if col.startswith("Smoking_")]
df[smoking_cols] = df[smoking_cols].astype(int)

# --- PINDAHKAN HEART.DISEASE KE KOLOM TERAKHIR ---
target = df['Heart.Disease']         # Simpan kolom target
df = df.drop(columns=['Heart.Disease'])  # Hapus dulu
df['Heart.Disease'] = target             # Tambahkan kembali di paling akhir

# Now you can work with the DataFrame 'df'
# For example, you can access columns, perform data analysis, etc.
#df = df.drop(df.columns[0], axis=1)
df
# Print the first few rows of the DataFrame
```

	Age	Gender	Cholesterol	Blood.Pressure	Exercise.Hours	Diabetes	Obesity	Blood.Sugar	Smoking_Current	Smoking_Former	Smoking_Never	Heart.Disease
0	75	0	228	119	1	0	1	119	1	0	0	1
1	48	1	204	165	5	0	0	70	1	0	0	0
2	53	1	234	91	3	0	1	196	0	0	1	1
3	69	0	192	90	4	1	0	107	1	0	0	0
4	62	0	172	163	6	1	0	183	0	0	1	0
...	...	...	...	...	...	...	...	...	...	...	...	...
995	56	0	269	111	5	1	1	120	0	0	1	1
996	78	0	334	145	6	0	0	196	0	0	1	1
997	79	1	151	179	4	0	1	189	0	0	1	0
998	60	0	326	151	8	1	0	174	0	1	0	1
999	53	1	226	116	6	0	1	161	1	0	0	1

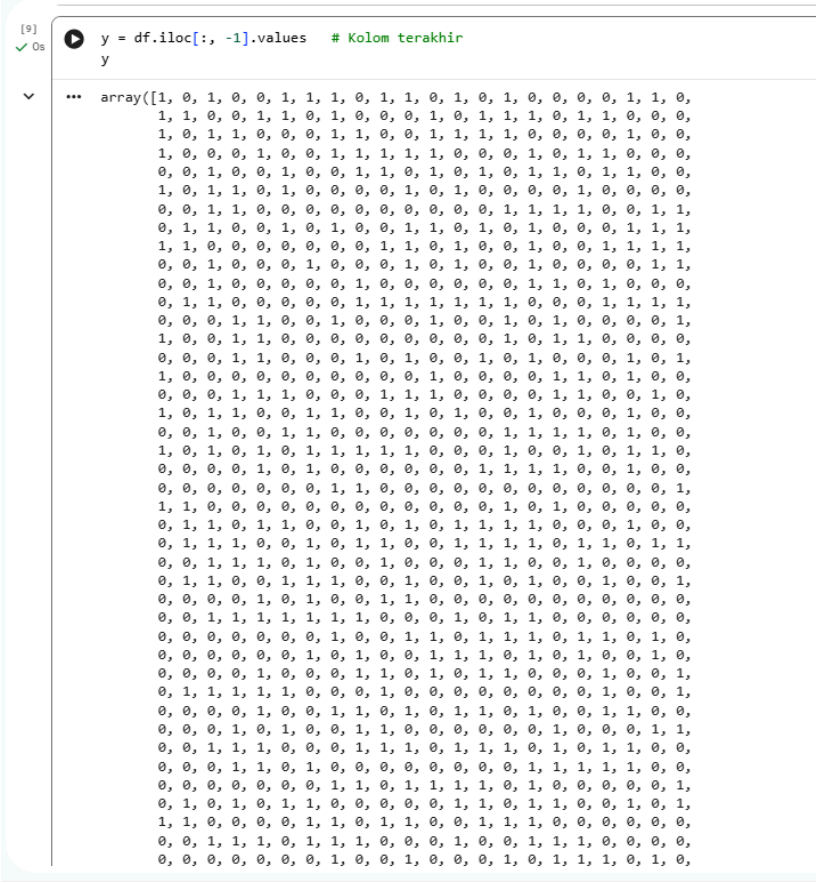
1000 rows × 12 columns

3. Mengambil semua kolom yang termasuk variabel X dengan menghilangkan kolom terakhir yang menjadi variabel y (target)

```
# Misalkan kolom terakhir sebagai target (y), dan sisanya sebagai fitur (X)
X = df.iloc[:, :-1].values  # Semua kolom kecuali yang terakhir
X
```

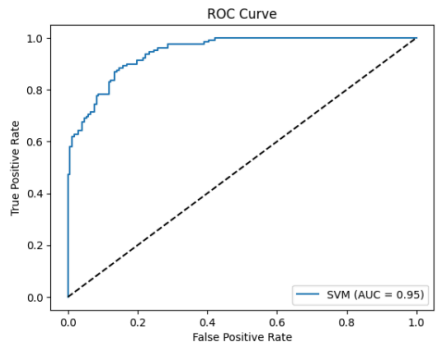
```
array([[ 75,   0, 228, ...,   1,   0,   0],
       [ 48,   1, 204, ...,   1,   0,   0],
       [ 53,   1, 234, ...,   0,   0,   1],
       ...,
       [ 79,   1, 151, ...,   0,   0,   1],
       [ 60,   0, 326, ...,   0,   1,   0],
       [ 53,   1, 226, ...,   1,   0,   0]])
```

4. Mengambil kolom target (variabel y) dari semua baris dataset, digunakan sebagai label dalam pemodelan klasifikasi



5. Melatih Model
    a. Data train 70%

```
Cross Validation Accuracy per fold: [0.8760683760683761, 0.8969957081545065, 0.9141630901287554]
Mean CV Accuracy: 0.895742391450546
Accuracy: 0.8567
Precision: 0.8772
Recall: 0.7752
ROC AUC: 0.9463
Confusion Matrix:
 [[157  14]
 [ 29 100]]
```

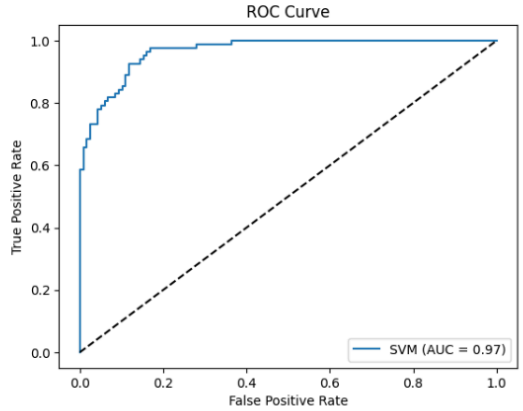



```
['scaler.pkl']
```

Dengan pembagian data train 70:30, sudah menghasilkan performa model yang baik dengan nilai ROC AUC sebesar 0.95 dan akurasi 0.856, tetapi nilai recall sebesar 0.775 yang masih dianggap cukup baik.

b. Data train 80%

```
Cross Validation Accuracy per fold: [0.8764044943820225, 0.850187265917603, 0.8947368421052632]
Mean CV Accuracy: 0.8737762008016295
Accuracy: 0.8750
Precision: 0.8701
Recall: 0.8171
ROC AUC: 0.9662
Confusion Matrix:
 [[108  10]
 [ 15  67]]
```

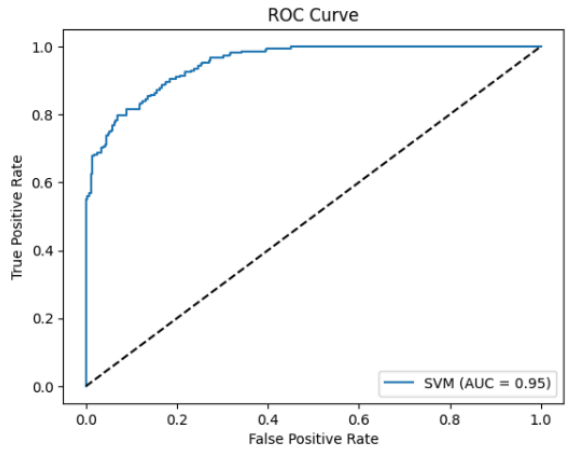



```
['scaler.pkl']
```

Dengan pembagian data train 80%, menghasilkan nilai ROC AUC sebesar 0.97 dengan akurasi 0.875, nilai recall pada model ini sebesar 0.817 yang dianggap baik

c. Data train 50%

```
Cross Validation Accuracy per fold: [0.8682634730538922, 0.8502994011976048, 0.9096385542168675]
Mean CV Accuracy: 0.8760671428227882
Accuracy: 0.8620
Precision: 0.9080
Recall: 0.7488
ROC AUC: 0.9510
Confusion Matrix:
 [[273  16]
 [ 53 158]]
```

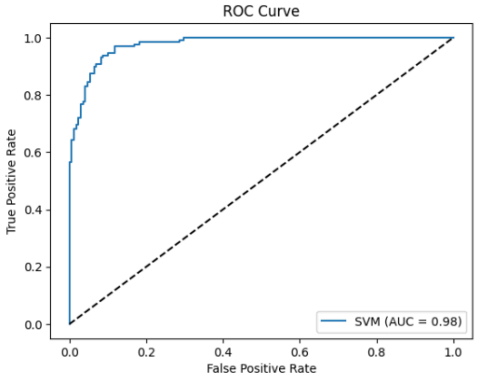



```
['scaler.pkl']
```

Dengan pembagian data train 50%, menghasilkan nilai ROC AUC sebesar 0.95 dengan akurasi 0.862, nilai recall pada model ini sebesar 0.748 yang mana paling rendah diantara model lainnya

6. Proses optimasi hyperparameter SVM
   a. Data train 70%

```
Fitting 3 folds for each of 81 candidates, totalling 243 fits
Best Parameters: {'C': 1, 'degree': 2, 'gamma': 0.1, 'kernel': 'rbf'}
Best ROC AUC Score: 0.9767
Accuracy: 0.9033
Precision: 0.9237
Recall: 0.8450
Confusion Matrix:
 [[162   9]
 [ 20 109]]
```



	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C	\
34	0.050978	0.000330	0.008758	0.000036	1.0	
43	0.060514	0.004838	0.010797	0.003013	1.0	
52	0.054285	0.002752	0.009089	0.000369	1.0	
30	0.043264	0.002321	0.005185	0.000426	1.0	
0	0.030575	0.001246	0.005783	0.000285	0.1	
22	0.068885	0.001530	0.012713	0.000395	0.1	
31	0.059653	0.008478	0.014769	0.006233	1.0	
18	0.032963	0.001394	0.005045	0.000103	0.1	
28	0.070238	0.001115	0.012233	0.000152	1.0	

```
    mean_fit_time  std_fit_time  mean_score_time  std_score_time  param_C  \
34       0.050978      0.000330         0.008758        0.000036      1.0
43       0.060514      0.004838         0.010797        0.003013      1.0
52       0.054285      0.002752         0.009089        0.000369      1.0
30       0.043264      0.002321         0.005185        0.000426      1.0
0        0.030575      0.001246         0.005783        0.000285      0.1
22       0.068885      0.001530         0.012713        0.000395      0.1
31       0.059653      0.008478         0.014769        0.006233      1.0
18       0.032963      0.001394         0.005045        0.000103      0.1
28       0.070238      0.001115         0.012233        0.000152      1.0
10       0.069004      0.001168         0.012129        0.000102      0.1
70       0.058820      0.013387         0.007098        0.000118     10.0
4        0.068644      0.000167         0.012503        0.000182      0.1
12       0.032914      0.001157         0.005015        0.000048      0.1

    param_degree  param_gamma param_kernel  \
34             2       0.100         rbf
43             3       0.100         rbf
52             4       0.100         rbf
30             2       0.010      linear
0              2       0.001      linear
22             4       0.010         rbf
31             2       0.010         rbf
18             4       0.001      linear
28             2       0.001         rbf
10             3       0.001         rbf
70             3       0.100         rbf
4              2       0.010         rbf
12             3       0.010      linear

                                                params  \
34       {'C': 1, 'degree': 2, 'gamma': 0.1, 'kernel': 'rbf'}
43       {'C': 1, 'degree': 3, 'gamma': 0.1, 'kernel': 'rbf'}
52       {'C': 1, 'degree': 4, 'gamma': 0.1, 'kernel': 'rbf'}
30    {'C': 1, 'degree': 2, 'gamma': 0.01, 'kernel': 'linear'}
0   {'C': 0.1, 'degree': 2, 'gamma': 0.001, 'kernel': 'linear'}
22      {'C': 0.1, 'degree': 4, 'gamma': 0.01, 'kernel': 'rbf'}
31        {'C': 1, 'degree': 2, 'gamma': 0.01, 'kernel': 'rbf'}
18  {'C': 0.1, 'degree': 4, 'gamma': 0.001, 'kernel': 'linear'}
28       {'C': 1, 'degree': 2, 'gamma': 0.001, 'kernel': 'rbf'}
10     {'C': 0.1, 'degree': 3, 'gamma': 0.001, 'kernel': 'rbf'}
70        {'C': 10, 'degree': 3, 'gamma': 0.1, 'kernel': 'rbf'}
4       {'C': 0.1, 'degree': 2, 'gamma': 0.01, 'kernel': 'rbf'}
12   {'C': 0.1, 'degree': 3, 'gamma': 0.01, 'kernel': 'linear'}
```

```
                                                params  \
34       {'C': 1, 'degree': 2, 'gamma': 0.1, 'kernel': 'rbf'}
43       {'C': 1, 'degree': 3, 'gamma': 0.1, 'kernel': 'rbf'}
52       {'C': 1, 'degree': 4, 'gamma': 0.1, 'kernel': 'rbf'}
30    {'C': 1, 'degree': 2, 'gamma': 0.01, 'kernel': 'linear'}
0   {'C': 0.1, 'degree': 2, 'gamma': 0.001, 'kernel': 'linear'}
22      {'C': 0.1, 'degree': 4, 'gamma': 0.01, 'kernel': 'rbf'}
31        {'C': 1, 'degree': 2, 'gamma': 0.01, 'kernel': 'rbf'}
18  {'C': 0.1, 'degree': 4, 'gamma': 0.001, 'kernel': 'linear'}
28       {'C': 1, 'degree': 2, 'gamma': 0.001, 'kernel': 'rbf'}
10     {'C': 0.1, 'degree': 3, 'gamma': 0.001, 'kernel': 'rbf'}
70        {'C': 10, 'degree': 3, 'gamma': 0.1, 'kernel': 'rbf'}
4       {'C': 0.1, 'degree': 2, 'gamma': 0.01, 'kernel': 'rbf'}
12   {'C': 0.1, 'degree': 3, 'gamma': 0.01, 'kernel': 'linear'}

    split0_test_score  split1_test_score  split2_test_score  mean_test_score  \
34           0.984278           0.975313           0.979058         0.979550
43           0.984278           0.975313           0.979058         0.979550
52           0.984278           0.975313           0.979058         0.979550
30           0.948941           0.948981           0.959534         0.952486
0            0.950265           0.946238           0.960242         0.952248
22           0.945984           0.949138           0.950323         0.948482
31           0.959449           0.954937           0.963077         0.959154
18           0.950265           0.946238           0.960242         0.952248
28           0.943104           0.945611           0.948040         0.945585
10           0.942870           0.945141           0.947567         0.945193
70           0.984978           0.972100           0.975122         0.977400
4            0.945984           0.949138           0.950323         0.948482
12           0.950265           0.946238           0.960242         0.952248

    std_test_score  rank_test_score
34        0.003676                1
43        0.003676                1
52        0.003676                1
30        0.004984               19
0         0.005887               37
22        0.001831               46
31        0.003329               13
18        0.005887               37
28        0.002015               49
10        0.001918               52
70        0.005499                4
4         0.001831               46
12        0.005887               37
```

Proses optimasi hyperparameter dengan pembagian data train 70:30 menghasilkan best parameter C : 1, degree : 2, gamma : 0.1, kernel : rbf, dengan best score ROC AUC 0.976, akurasi 0.903, precision 0.923, dan recall sebesar 0.845. Confusion matrix menunjukkan bahwa model hanya melewatkan 20 kasus positif dari total 129, dan membuat hanya 9 false positif.

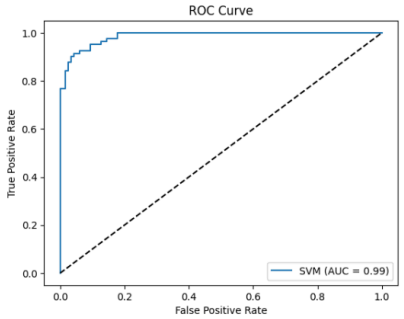
Confusion Matrix

Prediksi	**Tidak Sakit Jantung (0)**	**Sakit Jantung (1)**
0 (tidak sakit)	162 (TN)	20 (FN)
1 (sakit)	9 (FP)	109 (TP)

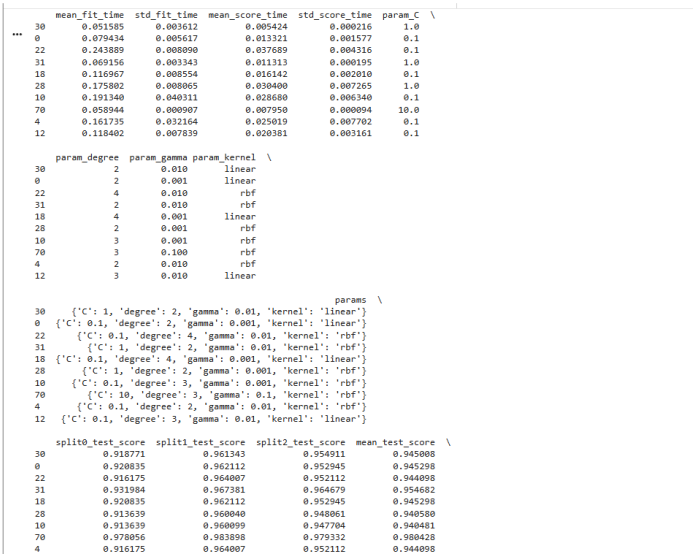
Kelas Positif (1) adalah nilai yang akan diprediksi.
TP (True Positive) = 109 → kasus yang diprediksi sakit dan memang sakit
FL (False Positive) = 9 → kasus yang diprediksi sakit tapi tidak benar-benar sakit.
Precision dalam Confusion Matrix = 0.9237, atau 92.37%, yang artinya, nilai presisi cukup tinggi dan model dapat memprediksi True Positive.
Recall dalam Confusion Matrix = 0.8450, yang artinya model berhasil mendeteksi 84% dari semua kasus sakit jantung (109 + 20 = 129), dan model gagal mendeteksi 20 kasus sakit jantung (20 False Negative).

b. Data train 80%

```
Fitting 3 folds for each of 81 candidates, totalling 243 fits
Best Parameters: {'C': 10, 'degree': 2, 'gamma': 0.1, 'kernel': 'rbf'}
Best ROC AUC Score: 0.9858
Accuracy: 0.9300
Precision: 0.9146
Recall: 0.9146
Confusion Matrix:
 [[111   7]
 [  7  75]]
```



	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C
30	0.051585	0.003612	0.005424	0.000216	1.0
0	0.079434	0.005617	0.013321	0.001577	0.1
22	0.243889	0.008090	0.037689	0.004316	0.1
31	0.069156	0.003343	0.011313	0.000195	1.0
18	0.116967	0.008554	0.016142	0.002010	0.1
28	0.175802	0.008065	0.030400	0.007265	1.0
10	0.191340	0.040311	0.028680	0.006340	0.1
70	0.058944	0.000907	0.007950	0.000094	10.0
4	0.161735	0.032164	0.025019	0.007702	0.1



	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C
30	0.051585	0.003612	0.005424	0.000216	1.0
0	0.079434	0.005617	0.013321	0.001577	0.1
22	0.243889	0.008090	0.037689	0.004316	0.1
31	0.069156	0.003343	0.011313	0.000195	1.0
18	0.116967	0.008554	0.016142	0.002010	0.1
28	0.175802	0.008065	0.030400	0.007265	1.0
10	0.191340	0.040311	0.028680	0.006340	0.1
70	0.058944	0.000907	0.007950	0.000094	10.0
4	0.161735	0.032164	0.025019	0.007702	0.1
12	0.118402	0.007839	0.020381	0.003161	0.1

	param_degree	param_gamma	param_kernel
30	2	0.010	linear
0	2	0.001	linear
22	4	0.010	rbf
31	2	0.010	rbf
18	4	0.001	linear
28	2	0.001	rbf
10	3	0.001	rbf
70	3	0.100	rbf
4	2	0.010	rbf
12	3	0.010	linear

	params
30	{'C': 1, 'degree': 2, 'gamma': 0.01, 'kernel': 'linear'}
0	{'C': 0.1, 'degree': 2, 'gamma': 0.001, 'kernel': 'linear'}
22	{'C': 0.1, 'degree': 4, 'gamma': 0.01, 'kernel': 'rbf'}
31	{'C': 1, 'degree': 2, 'gamma': 0.01, 'kernel': 'rbf'}
18	{'C': 0.1, 'degree': 4, 'gamma': 0.001, 'kernel': 'linear'}
28	{'C': 1, 'degree': 2, 'gamma': 0.001, 'kernel': 'rbf'}
10	{'C': 0.1, 'degree': 3, 'gamma': 0.001, 'kernel': 'rbf'}
70	{'C': 10, 'degree': 3, 'gamma': 0.1, 'kernel': 'rbf'}
4	{'C': 0.1, 'degree': 2, 'gamma': 0.01, 'kernel': 'rbf'}
12	{'C': 0.1, 'degree': 3, 'gamma': 0.01, 'kernel': 'linear'}

	split0_test_score	split1_test_score	split2_test_score	mean_test_score
30	0.918771	0.961343	0.954911	0.945008
0	0.920835	0.962112	0.952945	0.945298
22	0.916175	0.964007	0.952112	0.944098
31	0.931984	0.967381	0.964679	0.954682
18	0.920835	0.962112	0.952945	0.945298
28	0.913639	0.960040	0.948061	0.940580
10	0.913639	0.960099	0.947704	0.940481
70	0.978056	0.983898	0.979332	0.980428
4	0.916175	0.964007	0.952112	0.944098

	params
30	{'C': 1, 'degree': 2, 'gamma': 0.01, 'kernel': 'linear'}
0	{'C': 0.1, 'degree': 2, 'gamma': 0.001, 'kernel': 'linear'}
22	{'C': 0.1, 'degree': 4, 'gamma': 0.01, 'kernel': 'rbf'}
31	{'C': 1, 'degree': 2, 'gamma': 0.01, 'kernel': 'rbf'}
18	{'C': 0.1, 'degree': 4, 'gamma': 0.001, 'kernel': 'linear'}
28	{'C': 1, 'degree': 2, 'gamma': 0.001, 'kernel': 'rbf'}
10	{'C': 0.1, 'degree': 3, 'gamma': 0.001, 'kernel': 'rbf'}
70	{'C': 10, 'degree': 3, 'gamma': 0.1, 'kernel': 'rbf'}
4	{'C': 0.1, 'degree': 2, 'gamma': 0.01, 'kernel': 'rbf'}
12	{'C': 0.1, 'degree': 3, 'gamma': 0.01, 'kernel': 'linear'}

	split0_test_score	split1_test_score	split2_test_score	mean_test_score
30	0.918771	0.961343	0.954911	0.945008
0	0.920835	0.962112	0.952945	0.945298
22	0.916175	0.964007	0.952112	0.944098
31	0.931984	0.967381	0.964679	0.954682
18	0.920835	0.962112	0.952945	0.945298
28	0.913639	0.960040	0.948061	0.940580
10	0.913639	0.960099	0.947704	0.940481
70	0.978056	0.983898	0.979332	0.980428
4	0.916175	0.964007	0.952112	0.944098
12	0.920835	0.962112	0.952945	0.945298

	std_test_score	rank_test_score
30	0.018738	28
0	0.017698	19
22	0.020333	37
31	0.016087	13
18	0.017698	19
28	0.019668	49
10	0.019643	52
70	0.002508	1
4	0.020333	37
12	0.017698	19

Proses optimasi hyperparameter dengan pembagian data train 80:20 menghasilkan best parameter C : 1, degree : 2, gamma : 0.1, kernel : rbf, dengan best score ROC AUC 0.985, akurasi 0.93, precision 0.914, dan recall sebesar 0.914. Confusion matrix menunjukkan bahwa model hanya melewatkan 7 kasus positif dari total 82, dan membuat hanya 7 false positif

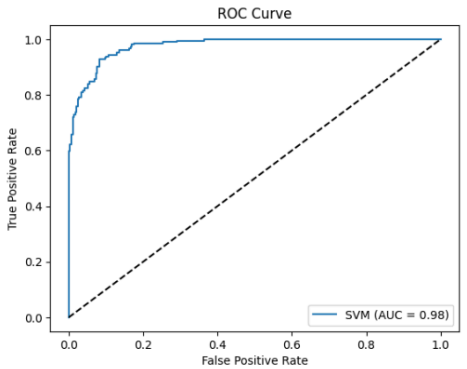
Confusion Matrix

Prediksi	**Tidak Sakit Jantung (0)**	**Sakit Jantung (1)**
0 (tidak sakit)	111 (TN)	7 (FN)
1 (sakit)	7 (FP)	75 (TP)
Total Sampel	111 + 7 = 118	7 + 75 = 82

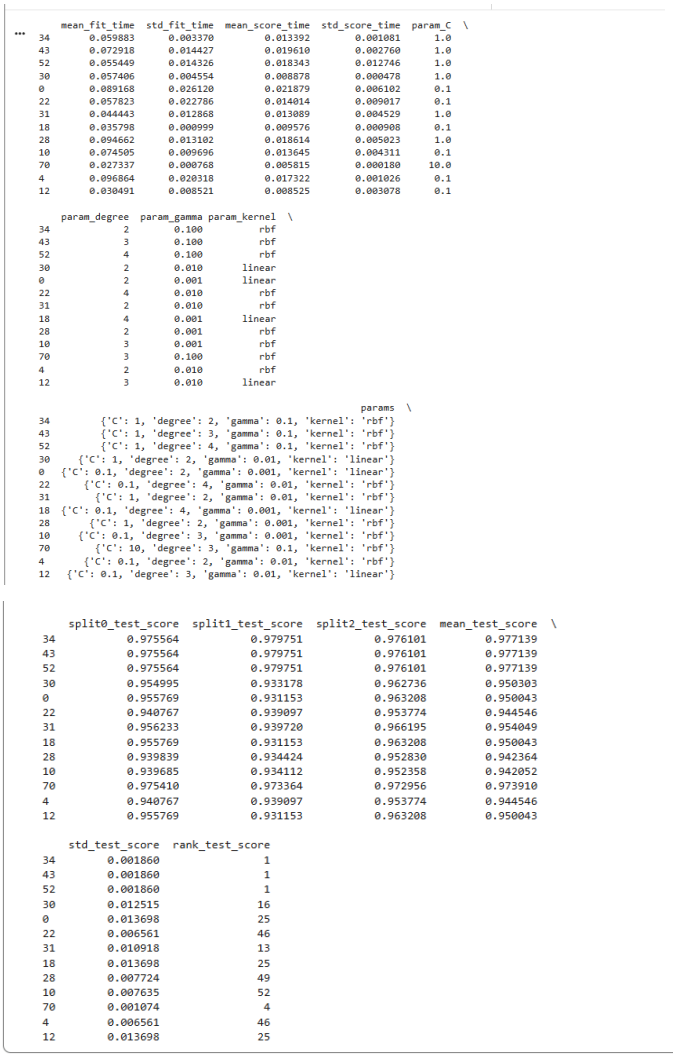
Kelas Positif (1) adalah nilai yang akan diprediksi.
TP (True Positive) = 111 → kasus yang diprediksi sakit dan memang sakit
FL (False Positive) = 7 → kasus yang diprediksi sakit tapi sebenarnya tidak benar-benar sakit (terlewat)
Precision dalam Confusion Matrix = 0.9146, atau 91%, yang artinya, nilai presisi juga cukup tinggi dan model dapat memprediksi True Positive.
Sementara nilai Recall dalam Confusion Matrix = 0.9146, yang artinya model berhasil mendeteksi 91% dari semua kasus sakit jantung (111 + 7 = 171), dan model gagal mendeteksi 7 kasus sakit jantung (7 False Negative).

c. Data train 50%

```
Fitting 3 folds for each of 81 candidates, totalling 243 fits
Best Parameters: {'C': 1, 'degree': 2, 'gamma': 0.1, 'kernel': 'rbf'}
Best ROC AUC Score: 0.9766
Accuracy: 0.9000
Precision: 0.9399
Recall: 0.8152
Confusion Matrix:
 [[278  11]
 [ 39 172]]
```



	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C	\
34	0.059883	0.003370	0.013392	0.001081	1.0	
43	0.072918	0.014427	0.019610	0.002760	1.0	
52	0.055449	0.014326	0.018343	0.012746	1.0	
30	0.057406	0.004554	0.008878	0.000478	1.0	
0	0.089168	0.026120	0.021879	0.006102	0.1	
22	0.057823	0.022786	0.014014	0.009017	0.1	
31	0.044443	0.012868	0.013089	0.004529	1.0	
18	0.035798	0.000999	0.009576	0.000908	0.1	
28	0.094662	0.013102	0.018614	0.005023	1.0	



	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C
34	0.059883	0.003370	0.013392	0.001081	1.0
43	0.072918	0.014427	0.019610	0.002760	1.0
52	0.055449	0.014326	0.018343	0.012746	1.0
30	0.057406	0.004554	0.008878	0.000478	1.0
0	0.089168	0.026120	0.021879	0.006102	0.1
22	0.057823	0.022786	0.014014	0.009017	0.1
31	0.044443	0.012868	0.013089	0.004529	1.0
18	0.035798	0.000999	0.009576	0.000908	0.1
28	0.094662	0.013102	0.018614	0.005023	1.0
10	0.074505	0.009696	0.013645	0.004311	0.1
70	0.027337	0.000768	0.005815	0.000180	10.0
4	0.096864	0.020318	0.017322	0.001026	0.1
12	0.030491	0.008521	0.008525	0.003078	0.1

	param_degree	param_gamma	param_kernel
34	2	0.100	rbf
43	3	0.100	rbf
52	4	0.100	rbf
30	2	0.010	linear
0	2	0.001	linear
22	4	0.010	rbf
31	2	0.010	rbf
18	4	0.001	linear
28	2	0.001	rbf
10	3	0.001	rbf
70	3	0.100	rbf
4	2	0.010	rbf
12	3	0.010	linear

	params
34	{'C': 1, 'degree': 2, 'gamma': 0.1, 'kernel': 'rbf'}
43	{'C': 1, 'degree': 3, 'gamma': 0.1, 'kernel': 'rbf'}
52	{'C': 1, 'degree': 4, 'gamma': 0.1, 'kernel': 'rbf'}
30	{'C': 1, 'degree': 2, 'gamma': 0.01, 'kernel': 'linear'}
0	{'C': 0.1, 'degree': 2, 'gamma': 0.001, 'kernel': 'linear'}
22	{'C': 0.1, 'degree': 4, 'gamma': 0.01, 'kernel': 'rbf'}
31	{'C': 1, 'degree': 2, 'gamma': 0.01, 'kernel': 'rbf'}
18	{'C': 0.1, 'degree': 4, 'gamma': 0.001, 'kernel': 'linear'}
28	{'C': 1, 'degree': 2, 'gamma': 0.001, 'kernel': 'rbf'}
10	{'C': 0.1, 'degree': 3, 'gamma': 0.001, 'kernel': 'rbf'}
70	{'C': 10, 'degree': 3, 'gamma': 0.1, 'kernel': 'rbf'}
4	{'C': 0.1, 'degree': 2, 'gamma': 0.01, 'kernel': 'rbf'}
12	{'C': 0.1, 'degree': 3, 'gamma': 0.01, 'kernel': 'linear'}

	split0_test_score	split1_test_score	split2_test_score	mean_test_score
34	0.975564	0.979751	0.976101	0.977139
43	0.975564	0.979751	0.976101	0.977139
52	0.975564	0.979751	0.976101	0.977139
30	0.954995	0.933178	0.962736	0.950303
0	0.955769	0.931153	0.963208	0.950043
22	0.940767	0.939097	0.953774	0.944546
31	0.956233	0.939720	0.966195	0.954049
18	0.955769	0.931153	0.963208	0.950043
28	0.939839	0.934424	0.952830	0.942364
10	0.939685	0.934112	0.952358	0.942052
70	0.975410	0.973364	0.972956	0.973910
4	0.940767	0.939097	0.953774	0.944546
12	0.955769	0.931153	0.963208	0.950043

	std_test_score	rank_test_score
34	0.001860	1
43	0.001860	1
52	0.001860	1
30	0.012515	16
0	0.013698	25
22	0.006561	46
31	0.010918	13
18	0.013698	25
28	0.007724	49
10	0.007635	52
70	0.001074	4
4	0.006561	46
12	0.013698	25

Proses optimasi hyperparameter dengan pembagian data train 50:50 menghasilkan best parameter C : 1, degree : 2, gamma : 0.1, kernel : rbf, dengan best score ROC AUC 0.976, akurasi 0.90, precision 0.939, dan recall sebesar 0.815. Confusion matrix menunjukkan bahwa model hanya melewatkan 39 kasus positif dari total 211, dan hanya membuat 11 false positif. Hal ini menunjukkan keseimbangan yang sangat baik antara sensitivitas dan spesifisitas.